DEVPILOT AI

# Database Persistence Guidelines

Relational modelling, SQL, transactions, and Java persistence

A retrieval-friendly engineering reference for Research and Coding Agents that design relational schemas, write queries, implement repositories, and review persistence changes.

| Document | Value |
| --- | --- |
| Scope | Relational modelling and persistence patterns for Java applications |
| Audience | Java engineers, reviewers, Research Agent and Coding Agent |
| Primary choices | Use JPA, JDBC, or both only when justified by actual use cases |
| Normative terms | MUST / SHOULD / MAY |
| RAG format | Literal Heading: and Section: prefixes in extracted text |

**Project-aware rule:** Inspect the existing POM, database engine, schema, repository conventions, migration tooling, and transaction boundaries before proposing changes. This guide does not require a specific persistence stack.

## Heading: Document Navigation — Contents and usage


1. Scope and decision rules
2. Relational modelling and table design
3. Keys, constraints, and data types
4. Normalization and relationship modelling
5. SQL naming and query readability
6. Parameterized SQL and injection prevention
7. Spring Data JPA versus JDBC
8. Entities, repositories, and service boundaries
9. Transactions and consistency
10. Indexes, pagination, and query performance
11. Database migrations and schema evolution
12. Connection configuration and credentials
13. Entity relationships and ORM pitfalls
14. Large result sets and N+1 queries
15. Repository implementation workflow
16. Good versus bad SQL and Java examples
17. Persistence review checklist


**How to use this guide:** Select the simplest persistence approach that meets the query, transaction, and domain requirements. Do not add JPA, JDBC, and a migration tool by default if the project does not need them.

# Heading: Scope and Decision Rules

## Section: Purpose and audience

This guide supports relational database design and Java persistence implementation. It covers schema design, SQL, Spring Data JPA, JDBC, transactions, performance, migrations, configuration, and repository testing considerations. It does not dictate a database engine or require every application to use an ORM.

## Section: MUST, SHOULD, and MAY

| Term | Meaning | Example |
|---|---|---|
| MUST | Required unless a documented project-specific constraint justifies an exception. | MUST bind untrusted values as query parameters. |
| SHOULD | Strong default with justified exceptions. | SHOULD add indexes for measured query patterns. |
| MAY | Optional when useful. | MAY use JPA for aggregate-oriented persistence. |

## Section: Inspect before changing persistence

- MUST inspect the existing schema, POM, database engine, repository interfaces, entity mappings, migrations, and transaction conventions.
- MUST preserve existing data and schema compatibility unless a migration is explicitly intended.
- MUST not introduce a second persistence technology without a concrete need.
- SHOULD make schema changes through the project's established migration process.
- MUST never assume a destructive migration is safe without understanding deployment order, backups, and data retention requirements.

# Heading: Relational Modelling and Table Design

## Section: Model facts and relationships

A relational schema should represent business facts with explicit entities, attributes, and relationships. Each table should have a clear purpose. Avoid storing multiple unrelated concepts in one table or encoding a variable-length list as comma-separated text when those values need independent validation, querying, or relationships.

## Section: Choose stable keys

Every table should have a primary key or a clearly documented key strategy. A surrogate key such as a generated UUID or numeric identity is often practical; a natural key may be appropriate when it is stable and genuinely identifies the entity. A primary key does not replace a unique constraint for business identifiers.

```
CREATE TABLE customer (
    id UUID PRIMARY KEY,
    email VARCHAR(320) NOT NULL,
    display_name VARCHAR(120) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    CONSTRAINT uq_customer_email UNIQUE (email)
);
```

This example uses PostgreSQL-compatible UUID and timestamp types. Email normalization and case-insensitive uniqueness require an explicit product rule and database strategy; a plain UNIQUE constraint may treat case differently from the business expectation.

## Section: Keep one fact in one place

Store a fact in one authoritative location where practical. If an order needs a customer reference, store a customer ID rather than copying the customer's current address into every order unless historical address snapshots are a deliberate requirement. Duplication can be valid when it captures historical facts or improves performance with controlled consistency.

## Section: Schema design questions

- What business entity or relationship does each table represent?
- Which fields are required, optional, unique, or immutable?
- What happens when a referenced record is deleted or deactivated?

- Which values need historical snapshots rather than live references?
- What is the expected row count and access pattern?
- What constraints should the database enforce even if application validation is bypassed?

# Heading: Keys, Constraints, and Data Types

## Section: Use database constraints for invariants

Database constraints protect data integrity across all writers, not just one Java code path. Use NOT NULL, UNIQUE, CHECK, PRIMARY KEY, and FOREIGN KEY constraints where they express real business invariants and are supported by the chosen database.

```
CREATE TABLE order_item (
    id UUID PRIMARY KEY,
    order_id UUID NOT NULL,
    product_id UUID NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity &gt; 0),
    unit_price NUMERIC(12, 2) NOT NULL CHECK (unit_price &gt;= 0),
    CONSTRAINT fk_item_order
        FOREIGN KEY (order_id) REFERENCES customer_order(id),
    CONSTRAINT fk_item_product
        FOREIGN KEY (product_id) REFERENCES product(id)
);
```

The referenced table and column names are illustrative. Ensure foreign keys point to the intended entities and align deletion behavior with the business lifecycle. Avoid cascading deletes that can erase business records unexpectedly.

## Section: Data types

| Data | Typical type | Guidance |
|---|---|---|
| Money/precise decimal | NUMERIC/DECIMAL | Avoid floating-point types for exact currency amounts; define scale and rounding rules. |
| Timestamp | Timestamp with time zone or database-specific equivalent | Choose instant versus local wall-clock semantics explicitly. |
| Identifier | UUID, integer, or stable natural key | Choose consistently with the system's key strategy. |
| Boolean | BOOLEAN | Avoid string flags such as 'Y'/'N' unless required by a legacy contract. |
| Text | VARCHAR with appropriate limit or TEXT | Set length constraints based on domain and database capabilities. |
| Status | Constrained text, reference table, or numeric code | Choose based on change frequency, querying, and interoperability needs. |

## Section: Nullability and defaults

Use NOT NULL when a value is required for a valid row. NULL should mean a deliberate unknown/not-applicable state, not an easy substitute for deciding what empty or absent means. Defaults are useful for stable database-level defaults but should not hide missing required application behavior.

## Section: Foreign-key delete behavior

Choose RESTRICT/NO ACTION, CASCADE, or SET NULL intentionally. For financial, audit, and order history, physical deletion may be inappropriate; soft deletion or retention policies may be required. Never assume cascade is safe merely because it simplifies cleanup.

# Heading: Normalization and Relationship Modelling

## Section: Normalization

Normalization reduces update anomalies by separating independently meaningful facts and using keys to relate them. Third normal form is a useful baseline for transactional data, but schema design still depends on real access patterns and historical semantics. Denormalization should be deliberate, measured, and accompanied by a consistency strategy.

### Section: One-to-many

```
CREATE TABLE customer_order (
    id UUID PRIMARY KEY,
    customer_id UUID NOT NULL REFERENCES customer(id),
    order_status VARCHAR(24) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL
);

CREATE TABLE order_line (
    id UUID PRIMARY KEY,
    order_id UUID NOT NULL REFERENCES customer_order(id),
    product_id UUID NOT NULL REFERENCES product(id),
    quantity INTEGER NOT NULL CHECK (quantity &gt; 0)
);
```

### Section: Many-to-many

A many-to-many relationship usually requires a join table when the relationship must be queried, constrained, or carry attributes.

```
CREATE TABLE project_member (
    project_id UUID NOT NULL REFERENCES project(id),
    user_id UUID NOT NULL REFERENCES app_user(id),
    member_role VARCHAR(32) NOT NULL,
    joined_at TIMESTAMP WITH TIME ZONE NOT NULL,
    PRIMARY KEY (project_id, user_id)
);
```

The composite primary key prevents duplicate membership. If the join row needs independent lifecycle, audit, or references, a surrogate key may be appropriate alongside a unique constraint on the pair.

### Section: Avoid comma-separated relationships

A column such as user_ids='1,4,9' prevents reliable foreign keys and makes filtering, updates, and uniqueness checks difficult. Normalize into a relationship table unless the field is intentionally an opaque external payload that is not queried as relational data.

## Heading: SQL Naming and Query Readability

### Section: Naming

- SHOULD use stable, consistent table and column naming, often snake_case in relational schemas.
- SHOULD choose names that communicate domain meaning and avoid reserved words.
- SHOULD use consistent singular/plural table conventions based on the existing schema.
- MUST use explicit column lists in production queries rather than SELECT * when stable projection matters.
- SHOULD name constraints and indexes when the database/project convention supports it.

### Section: Readable query design

```
-- Prefer explicit columns and clear predicates.
SELECT o.id, o.order_status, o.created_at
FROM customer_order AS o
WHERE o.customer_id = ?
  AND o.order_status = ?
ORDER BY o.created_at DESC
LIMIT ?;
```

The placeholder syntax shown is illustrative and differs by driver and database. Use the parameter syntax required by JDBC, JPA, or the selected database client. Keep filters and ordering explicit and reviewable.

### Section: Avoid hidden assumptions

Make ordering explicit when callers rely on it. SQL does not guarantee row order without ORDER BY. Define pagination order with a stable tie-breaker such as a unique key to avoid inconsistent page boundaries when multiple rows share the same timestamp.

## Heading: Parameterized SQL and Injection Prevention

### Section: Bind values; do not concatenate

All untrusted values used in SQL predicates or inserted/updated fields MUST be bound as parameters. Concatenating request input into SQL can let an attacker alter query structure. ORM APIs do not automatically make every query safe if developers build dynamic query strings.

```
// Unsafe: input can change the SQL expression.
String sql = "SELECT id FROM app_user WHERE email = '" + email + "'";

// Safe JDBC pattern.
String sql = "SELECT id FROM app_user WHERE email = ?";
try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, email);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            UUID id = rs.getObject("id", UUID.class);
            // Process the selected result.
        }
    }
}
```

### Section: Dynamic identifiers

Prepared-statement parameters represent values, not arbitrary SQL identifiers. If the user selects a sort field or direction, map that choice through a fixed allowlist and construct only the known-safe SQL fragment. Never append raw request text as a column name, table name, or ORDER BY expression.

```
String sortColumn = switch (requestedSort) {
    case "createdAt" -&gt; "created_at";
    case "status" -&gt; "order_status";
    default -&gt; throw new IllegalArgumentException("Unsupported sort field");
};
```

### Section: ORM query safety

Use named parameters or repository query binding for JPA queries. Do not assume JPQL/HQL string concatenation is safe. For bulk updates and native SQL, review parameter binding, transaction semantics, and persistence-context effects explicitly.

## Heading: Spring Data JPA Versus JDBC

### Section: Use JPA when object-relationship mapping helps

Spring Data JPA and an ORM can reduce boilerplate when the application has domain entities, relationships, and aggregate-oriented operations. JPA is useful when entity lifecycle, dirty checking, and repository abstractions fit the use case. It also introduces persistence-context behavior, lazy loading, flush semantics, and query-planning concerns that engineers must understand.

### Section: Use JDBC when SQL control is valuable

Spring JDBC is often a better fit for straightforward SQL, reporting queries, bulk operations, explicit projections, or cases where SQL behavior should be visible and predictable. It offers less object-relational automation but can make query behavior and selected columns clearer.

| Concern | JPA / Spring Data JPA | JDBC / Spring JDBC |
|---|---|---|
| Primary abstraction | Entities, persistence context, repository methods, JPQL/criteria | SQL statements, row mappers, explicit parameters |
| Good fit | Aggregate-oriented writes and mapped relationships | Complex SQL, reports, bulk work, explicit projections |
| Trade-offs | Lazy loading, N+1 queries, ORM lifecycle complexity | More mapping code; SQL and schema are more visible |
| Performance control | Requires awareness of fetch plans and generated SQL | Direct control over selected columns and query shape |
| Best reason to choose | Object model matches persistence operations | Explicit SQL behavior is valuable or ORM is unnecessary |

### Section: Do not add both without a reason

A project can use JPA for normal entity operations and JDBC for a measured performance-critical query, but that is a deliberate hybrid. Do not add JPA, JDBC, and multiple query frameworks just in case. Inspect current dependencies and use the simplest supported approach that meets requirements.

### Section: Repository interface examples

```
// Spring Data JPA style
public interface CustomerJpaRepository
        extends JpaRepository&lt;CustomerEntity, UUID&gt; {
    Optional&lt;CustomerEntity&gt; findByEmail(String email);
}

// Spring JDBC style
@Repository
public class CustomerJdbcRepository {
    private final JdbcTemplate jdbcTemplate;

    public CustomerJdbcRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public Optional&lt;CustomerSummary&gt; findSummary(UUID id) {
        return jdbcTemplate.query(
            "SELECT id, email FROM customer WHERE id = ?",
            (rs, rowNum) -&gt; new CustomerSummary(
                rs.getObject("id", UUID.class), rs.getString("email")),
            id
        ).stream().findFirst();
    }
}
```

These alternatives illustrate different designs; they are not intended to be pasted together without adapting the actual domain types and dependency setup.

# Heading: Entities, Repositories, and Service Boundaries

### Section: Entity design

JPA entities should follow the persistence provider's requirements for the project's version, including entity identity and construction rules. Keep entity mappings aligned with the schema and avoid putting HTTP serialization concerns in entity types. Entity methods may enforce domain invariants when the design intentionally uses entities as domain objects.

### Section: Repository responsibilities

- Repositories should expose useful persistence operations rather than arbitrary access to every table detail.
- Keep HTTP status codes and API DTO construction out of repositories.
- Use explicit projections for read models that do not require full entities.
- Do not expose a persistence session/entity manager throughout the application without a concrete reason.
- Keep repository transaction expectations clear and consistent with service boundaries.

### Section: Service responsibilities

Services coordinate a business operation across repository calls and other collaborators. Transaction boundaries commonly belong at the service/use-case layer when several persistence operations must succeed or fail together. Avoid making every repository call a separate transaction if the business operation needs atomicity across them.

### Section: Mapping boundaries

Map between persistence entities and API DTOs explicitly. When returning entities is unavoidable in a tightly controlled internal application, verify serialization, lazy relationships, sensitive fields, and compatibility implications rather than assuming it is harmless.

# Heading: Transactions and Consistency

### Section: Define the business transaction

A transaction groups database operations that must preserve a consistency invariant. Use a transaction around the smallest business operation that needs atomicity. In Spring, @Transactional behavior depends on proxying, propagation, rollback rules, and the transaction manager. Confirm the project's conventions before changing these semantics.

```
@Service
public class TransferService {
    private final AccountRepository accountRepository;

    public TransferService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
```

```
    }

    @Transactional
    public void transfer(UUID fromId, UUID toId, BigDecimal amount) {
        if (amount == null || amount.signum() &lt;= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
        Account source = accountRepository.findById(fromId)
                .orElseThrow(() -&gt; new AccountNotFoundException(fromId));
        Account target = accountRepository.findById(toId)
                .orElseThrow(() -&gt; new AccountNotFoundException(toId));
        source.debit(amount);
        target.credit(amount);
    }
}
```

This example assumes the repository returns managed entities and that the persistence setup flushes their changes within the transaction. A real transfer also needs concurrency control, sufficient-funds checks, currency rules, auditability, and idempotency appropriate to the business. Do not treat the snippet as a complete financial implementation.

### Section: Rollback behavior

By default, Spring transaction rollback behavior is generally triggered by unchecked exceptions and Errors, not every checked exception. Explicit rollback rules may be needed. Do not catch and suppress an exception inside a transaction if that would allow a partially completed business operation to commit.

### Section: Isolation and concurrency

Choose isolation and locking based on the invariant and database semantics. Optimistic locking with a version column can detect conflicting updates; pessimistic locking may be appropriate when contention and correctness require it. Avoid assuming that reading a row and later updating it is safe under concurrent requests.

### Section: External side effects

A database transaction does not automatically make email delivery, message publication, or remote HTTP calls atomic with the database. For reliable event publication, consider an outbox or another documented consistency pattern when justified. Avoid holding long database transactions open while waiting for slow remote services.

## Heading: Indexes, Pagination, and Query Performance

### Section: Index for access patterns

Indexes can speed up filters, joins, ordering, and uniqueness checks, but consume storage and add write cost. Add indexes based on real query patterns and query plans, not on every column by default. Composite index column order matters; align it with common predicates and ordering.

```
-- Example only: choose indexes from actual query plans.
CREATE INDEX idx_order_customer_created
    ON customer_order (customer_id, created_at DESC);
```

### Section: Pagination

For large collections, use bounded pagination or streaming patterns rather than loading every row into memory. Offset pagination is easy to implement but can become expensive for deep pages and may shift under concurrent inserts/deletes. Keyset pagination can be more stable and efficient when a suitable ordered key exists.

### Section: N+1 query patterns

An ORM may execute one query to load a list and then one additional query per item to load a relationship. Use query counts, SQL logs in controlled environments, fetch joins, entity graphs, or DTO projections where appropriate. Avoid globally eager-loading every relationship as a blanket fix.

### Section: Measure before optimizing

- Inspect slow-query logs and database execution plans.
- Select only the columns needed by the operation.
- Check index selectivity and write overhead.
- Bound result size and use stable ordering for pagination.
- Review connection pool saturation and transaction duration.

- Add caching only with a clear invalidation and consistency model.

# Heading: Database Migrations and Schema Evolution

## Section: Use versioned migrations

Schema changes should be repeatable, reviewable, and deployable through the project's established migration process. Common tools include Flyway and Liquibase, but neither is mandatory if the project uses another controlled process. Do not introduce a migration tool without checking the current stack.

## Section: Expand-and-contract for deployed systems

When application instances and database schema may be deployed at different times, prefer compatible migration sequences: add the new column/table or nullable field, deploy code that can coexist with the old shape, backfill if needed, switch reads/writes, and remove obsolete schema only after all consumers are migrated.

- MUST review migration ordering, rollback/forward-fix strategy, data volume, and locking impact.
- MUST not rely on automatic schema creation/update in production unless the organization explicitly approves that operating model.
- SHOULD use database constraints to enforce invariants after data is valid.
- SHOULD test migrations against a representative schema and data volume.
- MUST not include destructive DROP/DELETE operations without an explicit requirement and data-safety review.

## Section: Migration naming

Use the established tool's naming format and monotonically ordered version conventions. Migration files should be immutable after deployment in environments that track their checksums; create a new migration to correct a deployed migration rather than silently editing history.

# Heading: Connection Configuration and Credential Management

## Section: Externalize connection settings

Keep database URLs, usernames, pool sizes, and timeouts in configuration appropriate to the deployment. Credentials must come from approved secret delivery, not source control. Separate local development defaults from production configuration and avoid printing resolved credentials.

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      connection-timeout: 30000
```

This is an illustrative Spring Boot configuration. Confirm the project's pool implementation and supported property names. Do not add pool-specific settings unless the configured pool supports them.

## Section: Connection pool and timeouts

- Use a bounded connection pool sized for the database and workload; more connections are not always faster.
- Set connection, query, and transaction timeouts based on operational needs where supported.
- Monitor pool wait time, active/idle connections, failures, and long-running transactions.
- Ensure connections are returned promptly and JDBC resources are closed using try-with-resources or framework-managed APIs.
- Never log passwords or full credential-bearing JDBC URLs.

# Heading: Entity Relationships and Common ORM Pitfalls

## Section: Lazy loading and serialization

Lazy relationships may require an active persistence context. Returning entities from controllers can trigger unexpected queries or serialization failures. Map to DTOs within the intended transaction boundary and fetch the data needed for the response explicitly.

## Section: Cascade and orphan removal

Cascade settings propagate operations to related entities; orphan removal can delete a child when it is removed from a relationship. Use these only when lifecycle ownership is clear. Do not cascade REMOVE across relationships where the referenced object is shared or independently owned.

## Section: equals and hashCode

Entity equality is subtle when IDs are generated after construction and proxies are involved. Follow the project's established strategy; do not casually generate equals/hashCode across every mutable field or lazy relationship. Incorrect equality can break sets, maps, and persistence behavior.

## Section: N+1 and eager loading

Use targeted fetch plans and projections for each use case. Global EAGER relationships can produce large joins or unexpected data loading, while LAZY relationships require deliberate fetch planning. Verify generated SQL and query counts for high-volume paths.

## Section: Bulk operations

Bulk JPQL or SQL updates may bypass normal entity lifecycle callbacks and leave the persistence context stale. Flush/clear or otherwise manage state according to the actual operation and framework behavior. Test bulk changes with the project's transaction and caching configuration.

# Heading: Large Result Sets and Avoiding Unnecessary Queries

## Section: Process data incrementally

For exports, batch jobs, and large tables, avoid reading all rows into a List unless the maximum size is bounded and safe. Use paging, cursor/streaming APIs, database-side aggregation, or batch writes according to the driver/provider's capabilities. Close streams and result resources promptly.

## Section: Avoid per-row lookups

If code loops over rows and performs one query per row, evaluate joins, batch fetching, IN queries with bounded chunks, or projections. Preserve semantics and handle parameter-count limits when batching.

## Section: Read/write model separation

For complex reporting, a dedicated query model or JDBC projection can be simpler than loading a full aggregate graph. Choose a read model when query needs differ materially from write-domain operations, not as an automatic architecture requirement.

# Heading: Repository Implementation Workflow

## Section: Step-by-step workflow

```
INPUT: persistence requirement and repository context

1. Inspect schema, migrations, POM, database engine, and existing repository style.
2. Identify entities, relationships, constraints, and data-retention requirements.
3. Define the query/write operation and its transaction invariant.
4. Choose JPA, JDBC, or an existing hybrid based on the operation.
5. Bind all values; allowlist dynamic SQL identifiers.
6. Add constraints and migration changes using the established process.
7. Define transaction boundaries, rollback behavior, and concurrency controls.
8. Add repository/service tests for success, not-found, constraint, and concurrency cases.
9. Inspect generated SQL/query plans for important performance-sensitive paths.
10. Run tests and migration checks; inspect the final diff for data-loss risks.
11. Report actual verification and unresolved assumptions.
```

## Section: Design questions

- Can the database enforce this invariant with a constraint?

- Could two concurrent requests violate the rule?
- Does the operation need atomicity across multiple writes?
- Are external side effects incorrectly assumed to be transactional?
- Could this query load unbounded data or cause N+1 requests?
- Is the migration compatible with the rollout order and existing data?

# Heading: Good Versus Bad SQL and Java Persistence Examples

## Section: Avoid SELECT * for stable application queries

```
-- Less stable: schema additions can change returned columns.
SELECT * FROM customer WHERE id = ?;

-- Explicit projection.
SELECT id, email, display_name
FROM customer
WHERE id = ?;
```

## Section: Avoid per-row query loops

```
// Inefficient pattern: potentially one query per order.
for (Order order : orders) {
    Customer customer = customerRepository.findById(order.customerId())
            .orElseThrow();
    process(order, customer);
}

// Better: use a join/projection or bounded batch lookup where appropriate.
// Choose the implementation based on the actual query and data-access stack.
```

## Section: Avoid transaction swallowing

```
// Incorrect: an exception is hidden and a partial operation may commit.
@Transactional
public void updateTwoRecords() {
    repository.updateFirst();
    try {
        repository.updateSecond();
    } catch (RuntimeException ignored) {
        // The failure is lost.
    }
}

// Better: allow the failure to propagate or handle it with a valid recovery
// strategy that preserves the business invariant.
@Transactional
public void updateTwoRecords() {
    repository.updateFirst();
    repository.updateSecond();
}
```

## Section: Avoid dynamic SQL concatenation

```
// Incorrect
String sql = "SELECT id FROM customer ORDER BY " + requestedSort;

// Correct approach: allowlist the identifier, bind all values, and use
// the chosen data-access API's parameter mechanism.
```

## Section: Avoid accidental cascade deletion

Do not configure cascade REMOVE from an order to a shared product merely to simplify object graph cleanup. Deleting an order should not delete the catalog product. Model lifecycle ownership explicitly and test deletion behavior.

# Heading: Database Persistence Review Checklist

## Section: Schema and integrity

- Tables and relationships represent clear domain facts.
- Primary keys, unique constraints, foreign keys, nullability, and checks protect real invariants.
- Data types reflect precision, time semantics, size, and domain rules.
- Delete behavior and retention policies are deliberate.
- Migrations are versioned, reviewed, and compatible with deployment order.

## Section: Query and persistence design

- JPA or JDBC is selected for a concrete reason; no redundant stack was added.
- All untrusted values are parameterized; dynamic identifiers use strict allowlists.
- Repositories own persistence concerns; services define use-case transaction boundaries.
- Transaction rollback, isolation, locking, and concurrency risks are considered.
- DTOs/projections avoid unnecessary entity graph loading and sensitive field exposure.

## Section: Performance and operations

- Large collections are paginated or streamed with resource ownership handled correctly.
- Query plans and indexes match measured access patterns.
- N+1 queries, unnecessary columns, long transactions, and pool saturation are considered.
- Connection details are externalized and credentials are not committed or logged.
- Migration and rollback/forward-fix strategy is clear for production data.

## Section: Testing and reporting

- Repository tests cover success, empty/not-found, constraints, and relevant boundary cases.
- Transaction and concurrency behavior is tested where the invariant requires it.
- Migration tests run against a suitable database/schema where possible.
- Performance claims are based on measurements rather than assumptions.
- Reported commands and results reflect what was actually executed.

**End of PDF 4 — Database Persistence Guidelines.** The final document in the sequence is PDF 5, Java Testing and Quality Guidelines.

Document filename: `database-persistence-guidelines.pdf`